ChordDHT Node
Cloudflare Workers TypeScript 实现设计文档

版本	v1.0-CF · 草案
创建日期	2026 年 5 月 30 日
基于	ChordDHT Go 实现 v3.0（v1.0 + v2.0 + v3.0 设计文档）
适用范围	学习用途
文档类型	技术报告
受众	技术开发者

目录

第 1 章 Cloudflare 服务选型与限制

1.1 使用的 CF 服务一览

CF 服务	用途	必须付费?
Durable Objects (DO)	每个 DO 实例 = 一个 Chord 节点，持有全部状态	❌ 免费计划已包含（SQLite 后端）
DO Alarm	驱动 10s / 15s / 30s / 60s 周期维护，替代 Go `time.Ticker`	❌ 免费计划已包含
DO Storage (SQLite)	持久化 fingerTable、successorList、predecessor、状态机	❌ 免费计划 1 GB
Workers	HTTP 路由层，将请求转发到对应 DO 实例	❌ 免费计划已包含
Workers Secrets	存储 node_private_key、node_certificate、ca_public_key	❌ 免费计划已包含
Worker Cron	不使用——DO Alarm 完全替代	—
Workers KV	不使用——节点状态用 DO Storage，无需 KV	—

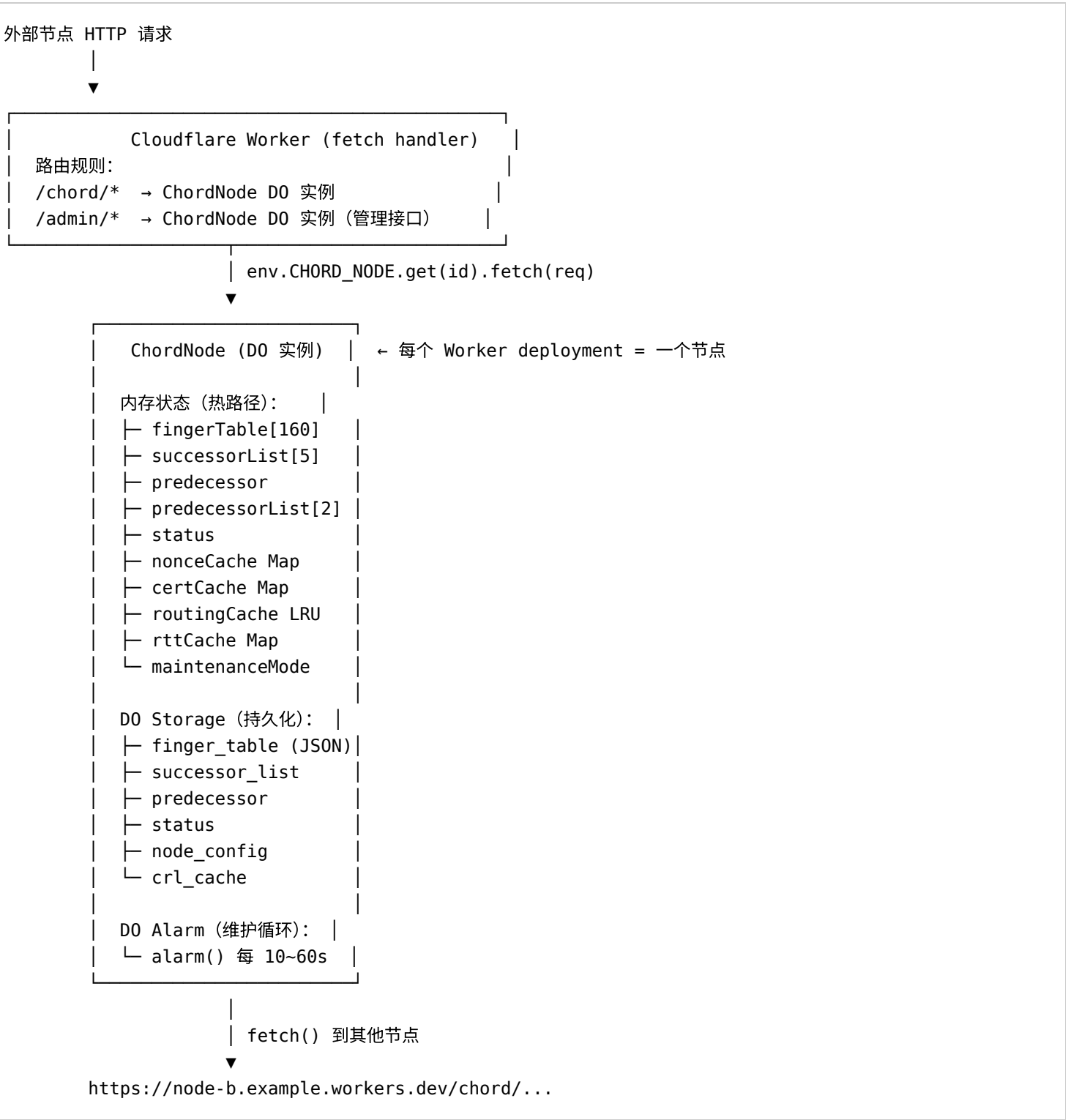
1.2 关键限制清单（查证版）

限制项	免费计划	付费计划（$5/月）	对本项目的影响
DO CPU 时间 / 请求	30 ms	30 s	⚠️ 免费计划 30 ms 不够用（Ed25519 验签约需 1–5 ms，批量 k=8 并发 fix_fingers 可能超限）；需付费计划
DO Alarm 最小间隔	无下限（毫秒级）	无下限	✅ v3.0 的 10 s fix_fingers 周期完全可行
DO Alarm CPU 时间	30 ms	30 s	⚠️ 同上，同一 CPU 限制，付费计划 30 s 绰绰有余
DO Storage 容量	1 GB	10 GB	✅ 节点状态极小（< 1 MB）
Worker subrequest 数 / 调用	50 次	1 000 次	⚠️ 免费计划 50 次对并行 fix_fingers 有影响；付费 1 000 次足够
Worker Cron 最小间隔	1 分钟	1 分钟	✅ 不使用 Cron，用 DO Alarm
DO 实例并发请求	串行（单线程）	串行（单线程）	✅ 天然消除所有 `sync.RWMutex`

> ⚠️ **结论：需要 Workers Paid 套餐（$5/月）**
> 原因是免费计划 DO CPU 时间仅 30 ms，Ed25519 签名验证 + 业务逻辑 + 并发 fetch 的总 CPU 耗时会超限。付费计划 30 s CPU / 请求，完全足够。

第 2 章 整体架构

2.1 架构概览图



2.2 部署模型说明

每个 Worker deployment 对应一个 Chord 节点。节点之间通过 HTTPS 互相通信，节点 URI 即 Worker 的 `workers.dev` 子域名或自定义域名，与 Go 实现的 `https://hostname` 完全对应。

```
https://node-a.example.workers.dev  →  部署 A（单个 DO 实例）
https://node-b.example.workers.dev  →  部署 B（单个 DO 实例）

node_id_A = SHA1("https://node-a.example.workers.dev")
node_id_B = SHA1("https://node-b.example.workers.dev")
```

每个节点具有以下独立性：

- 独立的 `wrangler.toml` 配置
- 独立的 Secrets（私钥、证书）
- 独立的 DO 实例

第 3 章 项目结构与 wrangler.toml

3.1 项目目录结构

```
chord-node/
├── wrangler.toml
├── src/
│   ├── index.ts              # Worker fetch handler（路由层）
│   ├── chord-node.ts         # ChordNode Durable Object（核心）
│   ├── chord/
│   │   ├── types.ts          # 所有类型定义
│   │   ├── ring.ts           # 160-bit 环形运算（BigInt）
│   │   ├── algorithms.ts     # find_successor、closest_preceding 等
│   │   ├── maintenance.ts    # stabilize、fix_fingers、check_predecessor
│   │   └── recovery.ts       # ISOLATED 多路径恢复
│   ├── auth/
│   │   ├── crypto.ts         # Web Crypto API 封装（Ed25519、SHA-1、SHA-256）
│   │   ├── cert.ts           # 证书结构、验证、签名消息构造
│   │   ├── signer.ts         # 请求签名生成
│   │   ├── verifier.ts       # 请求签名验证中间件
│   │   ├── nonce-cache.ts    # Nonce 去重缓存
│   │   └── cert-cache.ts     # 证书缓存（TTL=1hr）
│   ├── cache/
│   │   ├── lru-cache.ts      # LRU 缓存实现（路由缓存）
│   │   └── routing-cache.ts  # RoutingCache，含区间缓存
│   └── tracker/
│       └── tracker-client.ts # Tracker Client（本文档重点）
├── package.json
└── tsconfig.json
```

3.2 wrangler.toml 配置文件

```
name = "chord-node-a"
main = "src/index.ts"
compatibility_date = "2025-01-01"

[[durable_objects.bindings]]
name = "CHORD_NODE"
class_name = "ChordNode"

[[migrations]]
tag = "v1"
new_sqlite_classes = ["ChordNode"]

[vars]
NODE_URI = "https://node-a.example.workers.dev"
NODE_REGION = "us-east-1"
TRACKER_URL = "https://chord-tracker.example.workers.dev"
TRACKER_SEED_COUNT = "5"
AUTH_ENABLED = "true"
SUCCESSOR_LIST_SIZE = "5"
MAINTENANCE_ACTIVE_STABILIZE_MS = "15000"
MAINTENANCE_ACTIVE_FIX_FINGERS_MS = "10000"
MAINTENANCE_QUIET_STABILIZE_MS = "60000"
MAINTENANCE_QUIET_FIX_FINGERS_MS = "30000"
MAX_HOPS = "161"
HTTP_TIMEOUT_SAME_REGION_MS = "5000"
HTTP_TIMEOUT_CROSS_REGION_MS = "15000"
PARALLEL_LOOKUP_ENABLED = "false"

# Secrets（通过 wrangler secret put 设置，不写在此处）：
# NODE_PRIVATE_KEY_BASE64    Ed25519 私钥，32字节，base64url
# NODE_CERTIFICATE_JSON      完整 JSON 证书字符串
# CA_PUBLIC_KEY_BASE64       CA 公钥，32字节，base64url
# MANUAL_SEEDS               JSON 数组字符串，如 '["https://node-b..."]'
```

第 4 章 核心数据结构（TypeScript）

以下为 `src/chord/types.ts` 的完整类型定义，涵盖协议运行所需的全部接口与联合类型。

```
// src/chord/types.ts

export interface NodeInfo {
  node_id: string;           // 40位十六进制
  uri: string;               // 规范化 HTTPS URI
  status?: NodeStatus;
  joined_at?: string;        // ISO 8601 UTC
  certificate?: Certificate; // v2.0
  region?: string;           // v3.0
  version?: string;          // v3.0 "3.0"
  capabilities?: string[];   // v3.0
}

export type NodeStatus =
```

```
  | "INITIALIZING"
  | "JOINING"
  | "ACTIVE"
  | "LEAVING"
  | "ISOLATED";

export type MaintenanceMode =
  | "ACTIVE_MAINTENANCE"
  | "QUIET_MAINTENANCE";

export interface FingerEntry {
  index: number;             // 0..159
  start: string;             // (self.id + 2^i) mod 2^160, 40位hex
  node: NodeInfo;
  status: "ok" | "suspicious" | "unknown" | "failed";
  lastVerified: number;      // Unix ms
  valid: boolean;            // v3.0: false 优先重修
}

export interface RoutingCacheEntry {
  targetId: string;
  resultNode: NodeInfo;
  cachedAt: number;          // Unix ms
  intervalStart?: string;    // predecessor.id (exclusive)
  intervalEnd?: string;      // resultNode.id (inclusive)
}

export interface RTTEntry {
  avgRtt: number;            // ms, EWMA α=0.3
  sampleCount: number;
  updatedAt: number;         // Unix ms
}

export interface Certificate {
  version: 1;
  node_id: string;
  uri: string;
  public_key: string;        // base64url, 32字节 Ed25519 公钥
  issued_at: number;         // Unix 秒
  expires_at: number;        // Unix 秒
  signature: string;         // base64url, 64字节
}

export interface CRL {
  version: 1;
  updated_at: number;
  revoked: Array<{ node_id: string; revoked_at: number; reason: string }>;
  signature: string;
}

export interface PersistedState {
  nodeId: string;
  uri: string;
  status: NodeStatus;
  joinedAt: string | null;
  predecessor: NodeInfo | null;
  predecessorList: NodeInfo[];          // v3.0, 长度2
  successorList: NodeInfo[];            // v3.0, 长度r=5
```

```
  fingerTable: FingerEntry[];           // 160条
  nextFingerIndex: number;
  maintenanceMode: MaintenanceMode;
  lastChangeTime: number;               // Unix ms，用于ACTIVE/QUIET切换
  maintenanceCycles: number;
  region: string;
  crl: CRL | null;
  alarmScheduled: boolean;
}
```

4.1 类型结构说明

接口 / 类型	说明	版本引入
`NodeInfo`	节点基本身份信息，用于 finger table、successor list 等所有需要引用节点的场景	v1.0
`NodeStatus`	节点生命周期状态机，共 5 个状态	v1.0
`MaintenanceMode`	自适应维护模式（活跃 / 安静），影响 Alarm 触发频率	v3.0
`FingerEntry`	单条 finger 表项，含 valid 标志用于 v3.0 优先修复	v1.0 / v3.0
`RoutingCacheEntry`	路由缓存条目，支持区间命中优化	v3.0
`RTTEntry`	RTT 测量记录，使用 EWMA（α=0.3）平滑	v3.0
`Certificate`	节点身份证书，由 CA 签发，含 Ed25519 公钥	v2.0
`CRL`	证书吊销列表	v2.0
`PersistedState`	DO Storage 持久化的完整节点状态快照	v1.0

第 5 章 Durable Object：ChordNode 类

`ChordNode` 是整个系统的核心，位于 `src/chord-node.ts` 。每个 DO 实例代表一个 Chord 节点，其单线程执行模型天然消除了 Go 实现中的所有 `sync.RWMutex` 。

5.1 类骨架与内存状态

```
// src/chord-node.ts

import { DurableObject } from "cloudflare:workers";
import type { Env } from "./index";
import type {
  NodeInfo, NodeStatus, MaintenanceMode,
```

```
  FingerEntry, PersistedState
} from "./chord/types";
import { computeNodeId, normalizeUri } from "./chord/ring";
import { runMaintenanceCycle } from "./chord/maintenance";
import { AuthSigner } from "./auth/signer";
import { AuthVerifier } from "./auth/verifier";
import { RoutingCache } from "./cache/routing-cache";
import { NonceCache } from "./auth/nonce-cache";
import { CertCache } from "./auth/cert-cache";

export class ChordNode extends DurableObject<Env> {
  // ── 运行时内存状态（热路径，DO 单线程无需锁） ──────────────────
  private nodeId: string = "";
  private uri: string = "";
  private status: NodeStatus = "INITIALIZING";
  private joinedAt: string | null = null;
  private predecessor: NodeInfo | null = null;
  private predecessorList: NodeInfo[] = [];    // v3.0，长度2
  private successorList: NodeInfo[] = [];      // v3.0，长度r=5
  private fingerTable: FingerEntry[] = [];     // 160条
  private nextFingerIndex: number = 0;
  private maintenanceMode: MaintenanceMode = "ACTIVE_MAINTENANCE";
  private lastChangeTime: number = 0;
  private maintenanceCycles: number = 0;
  private region: string = "unknown";
  private alarmScheduled: boolean = false;

  // ── 内存缓存（DO 驱逐后重建，协议容忍） ──────────────────────
  private nonceCache: NonceCache = new NonceCache();
  private certCache: CertCache = new CertCache();
  private routingCache: RoutingCache = new RoutingCache(256);
  private rttCache: Map<string, { avgRtt: number; sampleCount: number; updatedAt: number }> = new
Map();
  private maintenanceModeActive: boolean = false;

  // ── 认证组件（从 Secrets 初始化） ──────────────────────────
  private signer: AuthSigner | null = null;
  private verifier: AuthVerifier | null = null;
  private authEnabled: boolean = false;

  constructor(ctx: DurableObjectState, env: Env) {
    super(ctx, env);
    this.ctx.blockConcurrencyWhile(() => this.initialize());
  }

  // ── initialize(): 从 DO Storage 恢复状态 ──────────────────
  private async initialize(): Promise<void> {
    // 1. 从 Storage 加载持久化状态
    const raw = await this.ctx.storage.get<PersistedState>("node_state");
    if (raw) {
      this.nodeId          = raw.nodeId;
      this.uri             = raw.uri;
      this.status          = raw.status;
      this.joinedAt        = raw.joinedAt;
      this.predecessor     = raw.predecessor;
      this.predecessorList = raw.predecessorList ?? [];
      this.successorList   = raw.successorList  ?? [];
      this.fingerTable     = raw.fingerTable    ?? [];
```

```
      this.nextFingerIndex = raw.nextFingerIndex ?? 0;
      this.maintenanceMode = raw.maintenanceMode ?? "ACTIVE_MAINTENANCE";
      this.lastChangeTime  = raw.lastChangeTime  ?? 0;
      this.maintenanceCycles = raw.maintenanceCycles ?? 0;
      this.region          = raw.region ?? "unknown";
      this.alarmScheduled  = raw.alarmScheduled ?? false;
    } else {
      // 首次启动：从 env 初始化
      this.uri    = normalizeUri(this.env.NODE_URI);
      this.nodeId = await computeNodeId(this.uri);
      this.region = this.env.NODE_REGION ?? "unknown";
      this.status = "INITIALIZING";
    }

    // 2. 初始化认证组件
    this.authEnabled = this.env.AUTH_ENABLED === "true";
    if (this.authEnabled) {
      this.signer   = await AuthSigner.create(this.env);
      this.verifier = await AuthVerifier.create(this.env);
    }

    // 3. 确保 Alarm 处于调度状态
    if (!this.alarmScheduled) {
      await this.scheduleAlarm(1000); // 1s 后首次触发
    }
  }

  // ── fetch()：主请求处理入口 ──────────────────────────────────────
  async fetch(request: Request): Promise<Response> {
    const url    = new URL(request.url);
    const path   = url.pathname;
    const method = request.method;

    // 免认证路径
    const publicPaths = ["/chord/ping", "/chord/info", "/chord/certificate"];
    const skipAuth    = publicPaths.some(p => path.startsWith(p));

    if (this.authEnabled && !skipAuth && this.verifier) {
      const authResult = await this.verifier.verify(request);
      if (!authResult.ok) {
        return new Response(
          JSON.stringify({ error: authResult.reason }),
          { status: 401, headers: { "Content-Type": "application/json" } }
        );
      }
    }

    // 路由分发
    switch (true) {
      // ── Chord 协议接口 ──────────────────────────────────────────
      case path === "/chord/ping"                    && method === "GET":
        return this.handlePing();
      case path === "/chord/info"                    && method === "GET":
        return this.handleInfo();
      case path === "/chord/find-successor"          && method === "POST":
        return this.handleFindSuccessor(request);
      case path === "/chord/predecessor"             && method === "GET":
        return this.handleGetPredecessor();
```

```
    case path === "/chord/notify"                  && method === "POST":
      return this.handleNotify(request);
    case path === "/chord/successor-list"          && method === "GET":
      return this.handleGetSuccessorList();
    case path === "/chord/certificate"             && method === "GET":
      return this.handleGetCertificate();

    // ── 管理接口 ──────────────────────────────────────────────
    case path === "/admin/join"                    && method === "POST":
      return this.handleJoin(request);
    case path === "/admin/leave"                   && method === "POST":
      return this.handleLeave();
    case path === "/admin/status"                  && method === "GET":
      return this.handleStatus();
    case path === "/admin/finger-table"            && method === "GET":
      return this.handleGetFingerTable();
    case path === "/admin/maintenance-mode"        && method === "POST":
      return this.handleSetMaintenanceMode(request);

    default:
      return new Response("Not Found", { status: 404 });
  }
}

// ── alarm(): DO Alarm 入口 ──────────────────────────────────
async alarm(): Promise<void> {
  this.alarmScheduled = false;
  try {
    await runMaintenanceCycle(this);
  } finally {
    await this.scheduleNextMaintenance();
  }
}

// ── scheduleAlarm() / scheduleNextMaintenance() ─────────────
private async scheduleAlarm(delayMs: number): Promise<void> {
  await this.ctx.storage.setAlarm(Date.now() + delayMs);
  this.alarmScheduled = true;
  await this.persistState();
}

private async scheduleNextMaintenance(): Promise<void> {
  const isActive = this.maintenanceMode === "ACTIVE_MAINTENANCE";
  // 使用 fix_fingers 周期作为下一次 Alarm 间隔
  const ms = isActive
    ? parseInt(this.env.MAINTENANCE_ACTIVE_FIX_FINGERS_MS  ?? "10000")
    : parseInt(this.env.MAINTENANCE_QUIET_FIX_FINGERS_MS   ?? "30000");
  await this.scheduleAlarm(ms);
}

// ── persistState(): 状态持久化 ──────────────────────────────
async persistState(): Promise<void> {
  const state: PersistedState = {
    nodeId:            this.nodeId,
    uri:               this.uri,
    status:            this.status,
    joinedAt:          this.joinedAt,
    predecessor:       this.predecessor,
```

```
      predecessorList:   this.predecessorList,
      successorList:     this.successorList,
      fingerTable:       this.fingerTable,
      nextFingerIndex:   this.nextFingerIndex,
      maintenanceMode:   this.maintenanceMode,
      lastChangeTime:    this.lastChangeTime,
      maintenanceCycles: this.maintenanceCycles,
      region:            this.region,
      crl:               null,
      alarmScheduled:    this.alarmScheduled,
    };
    await this.ctx.storage.put("node_state", state);
  }

  // ── onTopologyChange(): v3.0 自适应维护触发器 ──────────────────
  async onTopologyChange(): Promise<void> {
    this.lastChangeTime = Date.now();
    if (this.maintenanceMode !== "ACTIVE_MAINTENANCE") {
      this.maintenanceMode = "ACTIVE_MAINTENANCE";
      // 重新调度为活跃周期
      const ms = parseInt(this.env.MAINTENANCE_ACTIVE_FIX_FINGERS_MS ?? "10000");
      await this.scheduleAlarm(ms);
    }
  }

  // ── selfInfo(): 自身 NodeInfo ──────────────────────────────────
  selfInfo(): NodeInfo {
    return {
      node_id:  this.nodeId,
      uri:      this.uri,
      status:   this.status,
      region:   this.region,
      version:  "3.0",
      capabilities: ["chord-v3"],
    };
  }
}
```

第 6 章 DO Alarm：维护循环

> **核心机制说明**
>
> DO Alarm 是替代 Go `time.Ticker + goroutine` 的核心机制。节点自我续命：每次 `alarm()` 执行完后，调用 `setAlarm()` 设置下一次触发时间，形成永久循环，无需 Worker Cron 参与。

6.1 维护循环完整实现（src/chord/maintenance.ts）

```
// src/chord/maintenance.ts

import type { ChordNode } from "../chord-node";
```

```
import type { NodeInfo } from "./types";
import { inRange } from "./ring";

// ── runMaintenanceCycle(): 主维护循环入口 ──────────────────────────
export async function runMaintenanceCycle(node: ChordNode): Promise<void> {
  const status = node.getStatus();

  // 1. JOIN 流程
  if (status === "INITIALIZING" || status === "JOINING") {
    await runJoinFlow(node);
    return;
  }

  // 2. ISOLATED 恢复
  if (status === "ISOLATED") {
    await runIsolatedRecovery(node);
    return;
  }

  // 3. ACTIVE 正常维护周期（8步）
  if (status === "ACTIVE") {
    node.incrementMaintenanceCycles();

    // Step 1: check_predecessor
    await checkPredecessor(node);

    // Step 2: stabilize（每次均执行）
    await stabilize(node);

    // Step 3: fix_fingers（批量并行）
    const isActive = node.getMaintenanceMode() === "ACTIVE_MAINTENANCE";
    const batchSize = isActive ? 8 : 4;
    await fixFingersBatch(node, batchSize);

    // Step 4: 自适应维护模式切换
    const quietThreshold = 10; // 10次无拓扑变化 → QUIET
    const cycles = node.getMaintenanceCycles();
    const lastChange = node.getLastChangeTime();
    const now = Date.now();
    if (isActive && cycles > quietThreshold && (now - lastChange) > 120_000) {
      node.setMaintenanceMode("QUIET_MAINTENANCE");
    }

    // Step 5: 持久化（仅当状态有变化时）
    await node.persistState();
  }
}

// ── checkPredecessor(): v3.0 分层超时、快速崩溃检测 ────────────────
export async function checkPredecessor(node: ChordNode): Promise<void> {
  const pred = node.getPredecessor();
  if (!pred) return;

  const region      = node.getRegion();
  const predRegion  = pred.region ?? "unknown";
  const sameRegion  = region === predRegion;

  // v3.0: 同区域更短超时，跨区域更长超时
```

```
  const timeoutMs = sameRegion
    ? parseInt(node.env.HTTP_TIMEOUT_SAME_REGION_MS  ?? "5000")
    : parseInt(node.env.HTTP_TIMEOUT_CROSS_REGION_MS ?? "15000");

  try {
    const controller = new AbortController();
    const timer      = setTimeout(() => controller.abort(), timeoutMs);
    const resp = await node.rpcFetch(pred.uri + "/chord/ping", {
      method: "GET",
      signal: controller.signal,
    });
    clearTimeout(timer);

    if (!resp.ok) {
      node.markPredecessorFailed();
    }
  } catch {
    // 超时或网络错误 → 前驱失效
    node.markPredecessorFailed();
  }
}

// ── stabilize(): 稳定化协议 ──────────────────────────────────────
export async function stabilize(node: ChordNode): Promise<void> {
  const successorList = node.getSuccessorList();
  if (successorList.length === 0) return;

  let successor = successorList[0];

  // 1. 获取 successor 的前驱
  try {
    const resp = await node.rpcFetch(successor.uri + "/chord/predecessor", {
      method: "GET",
    });
    if (resp.ok) {
      const body = await resp.json<{ predecessor: NodeInfo | null }>();
      const x    = body.predecessor;

      // 2. 发现新节点插入：x 在 (self, successor) 区间内
      if (x && inRange(x.node_id, node.selfInfo().node_id, successor.node_id, false, false)) {
        successor = x;
        node.setSuccessorList([x, ...successorList.slice(0, 4)]);
        await node.onTopologyChange();
      }
    }
  } catch {
    // successor 无响应 → 切换到备用
    const backup = successorList.slice(1);
    if (backup.length > 0) {
      node.setSuccessorList(backup);
      await node.onTopologyChange();
    } else {
      node.setStatus("ISOLATED");
    }
    return;
  }

  // 3. 发送 notify（告知 successor 自己可能是其前驱）
```

```
  try {
    await node.rpcFetch(successor.uri + "/chord/notify", {
      method: "POST",
      body:   JSON.stringify({ node: node.selfInfo() }),
      headers: { "Content-Type": "application/json" },
    });
  } catch {
    // notify 失败不影响协议正确性
  }

  // 4. 刷新 successor_list
  try {
    const resp = await node.rpcFetch(successor.uri + "/chord/successor-list", {
      method: "GET",
    });
    if (resp.ok) {
      const body = await resp.json<{ successorList: NodeInfo[] }>();
      const r    = parseInt(node.env.SUCCESSOR_LIST_SIZE ?? "5");
      node.setSuccessorList([successor, ...body.successorList].slice(0, r));
    }
  } catch {
    // 忽略，保持现有列表
  }
}

// ── fixFingersBatch(): v3.0 批量并行修复 ──────────────────────────
export async function fixFingersBatch(node: ChordNode, batchSize: number): Promise<void> {
  const ft       = node.getFingerTable();
  const selfId   = node.selfInfo().node_id;
  const maxHops  = parseInt(node.env.MAX_HOPS ?? "161");

  // 优先修复 valid=false 的条目，其次按指数跨越顺序（高位优先）
  const invalidIndices = ft
    .filter(e => !e.valid)
    .map(e => e.index)
    .sort((a, b) => b - a); // 高位优先

  const validIndices = Array.from({ length: 160 }, (_, i) => i)
    .filter(i => !invalidIndices.includes(i))
    .sort((a, b) => b - a);

  const priorityQueue = [...invalidIndices, ...validIndices].slice(0, batchSize);

  // 并行查询
  const tasks = priorityQueue.map(async (i) => {
    const start = ft[i].start;
    try {
      const result = await node.findSuccessorIterative(start, maxHops);
      node.updateFingerEntry(i, result, true);
    } catch {
      node.updateFingerEntry(i, ft[i].node, false);
    }
  });

  await Promise.allSettled(tasks);
}
```

第 7 章 HTTP 路由层（Worker fetch handler）

> **设计原则**
> Worker 层极薄——只做一件事：将所有请求转发给唯一的 DO 实例（名为 `"singleton"`）。这是因为每个 `wrangler deploy` 对应一个节点，整个 Worker 里只存在一个 Chord 节点 DO 实例。

7.1 完整实现（src/index.ts）

```
// src/index.ts

export interface Env {
  CHORD_NODE: DurableObjectNamespace;
  NODE_URI: string;
  NODE_REGION: string;
  TRACKER_URL: string;
  TRACKER_SEED_COUNT: string;
  AUTH_ENABLED: string;
  SUCCESSOR_LIST_SIZE: string;
  NODE_PRIVATE_KEY_BASE64: string;
  NODE_CERTIFICATE_JSON: string;
  CA_PUBLIC_KEY_BASE64: string;
  MANUAL_SEEDS: string;
}

export { ChordNode } from "./chord-node";

export default {
  async fetch(request: Request, env: Env): Promise<Response> {
    const url  = new URL(request.url);
    const doId = env.CHORD_NODE.idFromName("singleton");
    const stub = env.CHORD_NODE.get(doId);
    return stub.fetch(request);
  },
} satisfies ExportedHandler<Env>;
```

7.2 路由设计说明

路径前缀	分类	说明
`/chord/ping`	协议接口（免认证）	存活探测，用于 checkPredecessor
`/chord/info`	协议接口（免认证）	返回节点基本信息
`/chord/certificate`	协议接口（免认证）	返回节点证书，用于验签
`/chord/find-successor`	协议接口（需认证）	迭代查找后继节点

路径前缀	分类	说明
/chord/predecessor	协议接口（需认证）	获取前驱节点，用于 stabilize
/chord/notify	协议接口（需认证）	通知前驱候选，用于 stabilize
/chord/successor-list	协议接口（需认证）	获取后继列表
/admin/join	管理接口	触发 Join 流程
/admin/leave	管理接口	触发 Graceful Leave
/admin/status	管理接口	查询节点运行状态
/admin/finger-table	管理接口	查看完整 finger 表
/admin/maintenance-mode	管理接口	手动切换维护模式

第 8 章 Chord 协议算法实现

8.1 160-bit 环形运算（src/chord/ring.ts）

```
// src/chord/ring.ts
// 所有 ID 均为 40 位十六进制字符串（160 bit），内部运算使用 BigInt

const RING_SIZE = 2n ** 160n;

export function hexToBigInt(hex: string): bigint {
  return BigInt("0x" + hex);
}

export function bigIntToHex(n: bigint): string {
  return n.toString(16).padStart(40, "0");
}

// 规范化 URI：去除末尾斜杠，转小写 scheme+host
export function normalizeUri(raw: string): string {
  const u = new URL(raw);
  return `${u.protocol}//${u.hostname}${u.port ? ":" + u.port : ""}`;
}

// 计算节点 ID：SHA-1(uri)
export async function computeNodeId(uri: string): Promise<string> {
  const encoded = new TextEncoder().encode(uri);
  const hashBuf = await crypto.subtle.digest("SHA-1", encoded);
  return Array.from(new Uint8Array(hashBuf))
    .map(b => b.toString(16).padStart(2, "0"))
    .join("");
}
```

```
// 顺时针距离：从 a 到 b 的顺时针弧长
export function clockwiseDistance(a: string, b: string): bigint {
  const ia = hexToBigInt(a);
  const ib = hexToBigInt(b);
  return ib >= ia ? ib - ia : RING_SIZE - ia + ib;
}

// inRange：id 是否在 (start, end] 区间内（半开半闭，顺时针）
export function inRange(
  id: string,
  start: string,
  end: string,
  startInclusive = false,
  endInclusive   = true
): boolean {
  const i  = hexToBigInt(id);
  const s  = hexToBigInt(start);
  const e  = hexToBigInt(end);

  const afterStart  = startInclusive ? i >= s : i > s;
  const beforeEnd   = endInclusive   ? i <= e : i < e;

  if (s < e) {
    return afterStart && beforeEnd;
  } else if (s > e) {
    // 跨越 0 点
    return afterStart || beforeEnd;
  } else {
    // s === e：全环（仅单节点情况）
    return true;
  }
}

// inRangeOpen：(start, end) 双开区间
export function inRangeOpen(id: string, start: string, end: string): boolean {
  return inRange(id, start, end, false, false);
}
```

8.2 find_successor 迭代模式（src/chord/algorithms.ts）

```
// src/chord/algorithms.ts

import type { ChordNode } from "../chord-node";
import type { NodeInfo } from "./types";
import { inRange, clockwiseDistance, hexToBigInt } from "./ring";

// ── localFindSuccessor()：本地快速判断 ──────────────────────────
export function localFindSuccessor(node: ChordNode, targetId: string): NodeInfo | null {
  const self      = node.selfInfo();
  const succList  = node.getSuccessorList();
  if (succList.length === 0) return null;

  const successor = succList[0];

  // target 在 (self, successor] 区间内 → successor 即是答案
  if (inRange(targetId, self.node_id, successor.node_id)) {
```

```
    return successor;
  }
  return null;
}

// ── computeScore(): v3.0 延迟感知路由评分 ──────────────────────────
// 综合评分 = ID接近度(0.6) + RTT分数(0.3) + Region亲和(0.1)
function computeScore(
  node: ChordNode,
  candidate: NodeInfo,
  targetId: string
): number {
  const dist     = clockwiseDistance(candidate.node_id, targetId);
  const maxDist  = 2n ** 160n;
  const idScore  = 1 - Number(dist * 1_000_000n / maxDist) / 1_000_000;

  const rtt      = node.getRtt(candidate.uri);
  const rttScore = rtt === null ? 0.5 : Math.max(0, 1 - rtt / 1000);

  const region      = node.getRegion();
  const candRegion  = candidate.region ?? "unknown";
  const regionScore = region === candRegion ? 1.0 : 0.0;

  return idScore * 0.6 + rttScore * 0.3 + regionScore * 0.1;
}

// ── closestPrecedingNode(): v3.0 延迟感知，综合评分选最优 ──────────
export function closestPrecedingNode(node: ChordNode, targetId: string): NodeInfo {
  const self     = node.selfInfo();
  const ft       = node.getFingerTable();

  // 候选集：finger table 中在 (self, target) 区间内的节点
  const candidates: NodeInfo[] = [];
  for (let i = 159; i >= 0; i--) {
    const entry = ft[i];
    if (entry.status === "failed" || !entry.valid) continue;
    if (inRange(entry.node.node_id, self.node_id, targetId, false, false)) {
      candidates.push(entry.node);
    }
  }

  if (candidates.length === 0) return self;

  // 按评分排序，取最高分
  candidates.sort((a, b) => computeScore(node, b, targetId) - computeScore(node, a, targetId));
  return candidates[0];
}

// ── findSuccessorIterative(): 迭代循环，含路由缓存、环路检测 ────────
export async function findSuccessorIterative(
  node: ChordNode,
  targetId: string,
  maxHops: number
): Promise<NodeInfo> {
  // 1. 路由缓存命中
  const cached = node.routingCache.lookup(targetId);
  if (cached) return cached;
```

```
  // 2. 本地快速判断
  const local = localFindSuccessor(node, targetId);
  if (local) {
    node.routingCache.store(targetId, local, node.selfInfo().node_id, local.node_id);
    return local;
  }

  // 3. 迭代查找
  let current     = node.selfInfo();
  const visited   = new Set<string>();
  let hops        = 0;

  while (hops < maxHops) {
    // 环路检测
    if (visited.has(current.node_id)) {
      throw new Error(`Routing loop detected at ${current.node_id} after ${hops} hops`);
    }
    visited.add(current.node_id);
    hops++;

    // 向 current 查询
    let resp: Response;
    try {
      resp = await node.rpcFetch(current.uri + "/chord/find-successor", {
        method:  "POST",
        body:    JSON.stringify({ target_id: targetId }),
        headers: { "Content-Type": "application/json" },
      });
    } catch {
      // 节点失联 → 标记失败，退出
      node.markNodeFailed(current);
      throw new Error(`Node unreachable: ${current.uri}`);
    }

    if (!resp.ok) throw new Error(`find-successor HTTP ${resp.status}`);

    const body = await resp.json<{ found: boolean; successor?: NodeInfo; next_hop?: NodeInfo }>();

    if (body.found && body.successor) {
      const result = body.successor;
      node.routingCache.store(targetId, result, current.node_id, result.node_id);
      return result;
    }

    if (body.next_hop) {
      current = body.next_hop;
    } else {
      throw new Error("Invalid find-successor response: no found and no next_hop");
    }
  }

  throw new Error(`find-successor exceeded maxHops=${maxHops}`);
}
```

第 9 章 v2.0 身份验证：Web Crypto API

9.1 crypto.ts — Ed25519 / SHA 封装

```
// src/auth/crypto.ts
// Web Crypto API 封装，仅使用原生 SubtleCrypto，无第三方依赖

// ── Ed25519 密钥导入 ──────────────────────────────────────────
export async function importPrivateKey(base64url: string): Promise<CryptoKey> {
  const raw = base64urlToBytes(base64url);
  // Ed25519 私钥为 32 字节原始格式
  return crypto.subtle.importKey(
    "raw",
    raw,
    { name: "Ed25519" },
    false,
    ["sign"]
  );
}

export async function importPublicKey(base64url: string): Promise<CryptoKey> {
  const raw = base64urlToBytes(base64url);
  return crypto.subtle.importKey(
    "raw",
    raw,
    { name: "Ed25519" },
    true,
    ["verify"]
  );
}

// ── 签名 / 验签 ──────────────────────────────────────────────
export async function signEd25519(
  key: CryptoKey,
  message: BufferSource
): Promise<string> {
  const sig = await crypto.subtle.sign("Ed25519", key, message);
  return bytesToBase64url(new Uint8Array(sig));
}

export async function verifyEd25519(
  key: CryptoKey,
  signature: string,
  message: BufferSource
): Promise<boolean> {
  const sigBytes = base64urlToBytes(signature);
  return crypto.subtle.verify("Ed25519", key, sigBytes, message);
}

// ── 哈希 ─────────────────────────────────────────────────────
export async function sha256Hex(input: string): Promise<string> {
  const encoded = new TextEncoder().encode(input);
  const hash    = await crypto.subtle.digest("SHA-256", encoded);
  return Array.from(new Uint8Array(hash))
    .map(b => b.toString(16).padStart(2, "0"))
```

```
    .join("");
}

export async function sha1Hex(input: string): Promise<string> {
  const encoded = new TextEncoder().encode(input);
  const hash    = await crypto.subtle.digest("SHA-1", encoded);
  return Array.from(new Uint8Array(hash))
    .map(b => b.toString(16).padStart(2, "0"))
    .join("");
}

// ── Base64url 工具 ──────────────────────────────────────────
export function base64urlToBytes(b64: string): Uint8Array {
  const padded = b64.replace(/-/g, "+").replace(/_/g, "/")
    + "=".repeat((4 - b64.length % 4) % 4);
  const binary = atob(padded);
  return Uint8Array.from(binary, c => c.charCodeAt(0));
}

export function bytesToBase64url(bytes: Uint8Array): string {
  return btoa(String.fromCharCode(...bytes))
    .replace(/\+/g, "-")
    .replace(/\//g, "_")
    .replace(/=/g, "");
}
```

9.2 signer.ts — 请求签名生成

```
// src/auth/signer.ts
// 签名消息规范 v2.0 §4.2

import type { Env } from "../index";
import { importPrivateKey, signEd25519 } from "./crypto";

export class AuthSigner {
  private constructor(
    private readonly privateKey: CryptoKey,
    private readonly nodeId: string,
    private readonly certificate: string
  ) {}

  static async create(env: Env): Promise<AuthSigner> {
    const privKey = await importPrivateKey(env.NODE_PRIVATE_KEY_BASE64);
    const cert    = JSON.parse(env.NODE_CERTIFICATE_JSON);
    return new AuthSigner(privKey, cert.node_id, env.NODE_CERTIFICATE_JSON);
  }

  // signRequest(): 生成签名 Headers
  async signRequest(
    method: string,
    url: string,
    body: string | null
  ): Promise<Record<string, string>> {
    const timestamp = Math.floor(Date.now() / 1000).toString();
    const nonce     = crypto.randomUUID().replace(/-/g, "");
```

```
    // 签名消息：METHOD\nURL\nTIMESTAMP\nNONCE[\nSHA256(body)]
    const parts = [method.toUpperCase(), url, timestamp, nonce];
    if (body) {
      const bodyHash = await sha256Hex(body);
      parts.push(bodyHash);
    }
    const message   = parts.join("\n");
    const msgBytes  = new TextEncoder().encode(message);
    const signature = await signEd25519(this.privateKey, msgBytes);

    return {
      "X-Chord-Node-Id":  this.nodeId,
      "X-Chord-Timestamp": timestamp,
      "X-Chord-Nonce":    nonce,
      "X-Chord-Signature": signature,
    };
  }
}

async function sha256Hex(input: string): Promise<string> {
  const encoded = new TextEncoder().encode(input);
  const hash    = await crypto.subtle.digest("SHA-256", encoded);
  return Array.from(new Uint8Array(hash))
    .map(b => b.toString(16).padStart(2, "0"))
    .join("");
}
```

9.3 verifier.ts — 请求签名验证中间件

```
// src/auth/verifier.ts

import type { Env } from "../index";
import { importPublicKey, verifyEd25519 } from "./crypto";
import { base64urlToBytes } from "./crypto";
import type { Certificate } from "../chord/types";

export interface VerifyResult {
  ok: boolean;
  reason?: string;
}

export class AuthVerifier {
  private readonly caPublicKey: CryptoKey;
  private readonly nonceCache: Map<string, number> = new Map();
  private readonly certCache:  Map<string, { cert: Certificate; key: CryptoKey; cachedAt: number }
> = new Map();

  private constructor(caPublicKey: CryptoKey) {
    this.caPublicKey = caPublicKey;
  }

  static async create(env: Env): Promise<AuthVerifier> {
    const caKey = await importPublicKey(env.CA_PUBLIC_KEY_BASE64);
    return new AuthVerifier(caKey);
  }
```

```
// verify(): 7步验证流程
async verify(request: Request): Promise<VerifyResult> {
  // Step 1: Header 完整性检查
  const nodeId    = request.headers.get("X-Chord-Node-Id");
  const timestamp = request.headers.get("X-Chord-Timestamp");
  const nonce     = request.headers.get("X-Chord-Nonce");
  const signature = request.headers.get("X-Chord-Signature");

  if (!nodeId || !timestamp || !nonce || !signature) {
    return { ok: false, reason: "Missing auth headers" };
  }

  // Step 2: 时间窗口（±5 分钟）
  const now = Math.floor(Date.now() / 1000);
  const ts  = parseInt(timestamp);
  if (Math.abs(now - ts) > 300) {
    return { ok: false, reason: "Timestamp out of window" };
  }

  // Step 3: Nonce 去重
  const nonceKey = `${nodeId}:${nonce}`;
  if (this.nonceCache.has(nonceKey)) {
    return { ok: false, reason: "Duplicate nonce" };
  }

  // Step 4: 获取并验证证书
  let pubKey: CryptoKey;
  const cached = this.certCache.get(nodeId);
  if (cached && Date.now() - cached.cachedAt < 3_600_000) {
    pubKey = cached.key;
  } else {
    // 从远端拉取证书（仅限非公开路径）
    const certResp = await fetch(
      `https://${new URL(request.url).hostname}/chord/certificate`
    );
    if (!certResp.ok) return { ok: false, reason: "Cannot fetch certificate" };

    const cert = await certResp.json<Certificate>();
    const validCert = await this.validateCertificate(cert, nodeId);
    if (!validCert.ok) return { ok: false, reason: validCert.reason };

    pubKey = await importPublicKey(cert.public_key);
    this.certCache.set(nodeId, { cert, key: pubKey, cachedAt: Date.now() });
  }

  // Step 5: 请求签名验证
  const body      = request.body ? await request.clone().text() : null;
  const parts     = [request.method.toUpperCase(), request.url, timestamp, nonce];
  if (body) {
    const bodyHash = await sha256Hex(body);
    parts.push(bodyHash);
  }
  const message  = parts.join("\n");
  const msgBytes = new TextEncoder().encode(message);
  const valid    = await verifyEd25519(pubKey, signature, msgBytes);

  if (!valid) return { ok: false, reason: "Invalid signature" };
```

```
    // Step 6: Nonce 入缓存（有效期 10 分钟）
    this.nonceCache.set(nonceKey, Date.now());
    // 懒清理过期 nonce
    if (this.nonceCache.size > 10_000) {
      const cutoff = Date.now() - 600_000;
      for (const [k, t] of this.nonceCache) {
        if (t < cutoff) this.nonceCache.delete(k);
      }
    }

    return { ok: true };
  }

  // validateCertificate(): 格式检查、node_id 一致性、有效期、CA 签名验证
  private async validateCertificate(
    cert: Certificate,
    expectedNodeId: string
  ): Promise<VerifyResult> {
    // 格式检查
    if (!cert.version || !cert.node_id || !cert.uri || !cert.public_key
        || !cert.issued_at || !cert.expires_at || !cert.signature) {
      return { ok: false, reason: "Malformed certificate" };
    }

    // node_id 一致性
    if (cert.node_id !== expectedNodeId) {
      return { ok: false, reason: "Certificate node_id mismatch" };
    }

    // 有效期
    const now = Math.floor(Date.now() / 1000);
    if (now < cert.issued_at || now > cert.expires_at) {
      return { ok: false, reason: "Certificate expired or not yet valid" };
    }

    // CA 签名验证
    // 签名消息: VERSION\nNODE_ID\nURI\nPUBLIC_KEY\nISSUED_AT\nEXPIRES_AT
    const message = [
      cert.version, cert.node_id, cert.uri,
      cert.public_key, cert.issued_at, cert.expires_at
    ].join("\n");
    const msgBytes = new TextEncoder().encode(message);
    const valid    = await verifyEd25519(this.caPublicKey, cert.signature, msgBytes);

    if (!valid) return { ok: false, reason: "Certificate CA signature invalid" };
    return { ok: true };
  }
}

async function sha256Hex(input: string): Promise<string> {
  const encoded = new TextEncoder().encode(input);
  const hash    = await crypto.subtle.digest("SHA-256", encoded);
  return Array.from(new Uint8Array(hash))
    .map(b => b.toString(16).padStart(2, "0"))
    .join("");
}
```

第 10 章 v3.0 优化实现

10.1 LRU 路由缓存（src/cache/routing-cache.ts）

`RoutingCache` 是 v3.0 的核心性能优化，支持精确匹配和区间命中两种查找模式，显著减少 find_successor 的网络跳数。

```
// src/cache/routing-cache.ts

import type { NodeInfo } from "../chord/types";
import { hexToBigInt } from "../chord/ring";

interface CacheEntry {
  targetId:      string;
  resultNode:    NodeInfo;
  cachedAt:      number;       // Unix ms
  intervalStart: string | undefined;   // predecessor.id (exclusive)
  intervalEnd:   string | undefined;   // resultNode.id (inclusive)
}

const TTL_MS = 60_000; // 1 分钟

export class RoutingCache {
  private readonly capacity: number;
  // 使用 Map 维护 LRU 顺序（插入顺序 = 最近访问在末尾）
  private readonly map: Map<string, CacheEntry> = new Map();
  // 区间索引：resultNode.id → CacheEntry[]
  private readonly intervalIndex: Map<string, CacheEntry[]> = new Map();

  constructor(capacity = 256) {
    this.capacity = capacity;
  }

  // lookup()：精确匹配 + 区间命中
  lookup(targetId: string): NodeInfo | null {
    const now = Date.now();

    // 1. 精确匹配
    const exact = this.map.get(targetId);
    if (exact) {
      if (now - exact.cachedAt > TTL_MS) {
        this.evict(targetId);
        return null;
      }
      this.touch(targetId, exact);
      return exact.resultNode;
    }

    // 2. 区间命中：遍历区间索引，检查 targetId 是否在 (start, end] 内
    const targetBig = hexToBigInt(targetId);
    for (const [endId, entries] of this.intervalIndex) {
      for (const entry of entries) {
        if (!entry.intervalStart || !entry.intervalEnd) continue;
```

```
      if (now - entry.cachedAt > TTL_MS) {
        this.evict(entry.targetId);
        continue;
      }

      const startBig = hexToBigInt(entry.intervalStart);
      const endBig   = hexToBigInt(entry.intervalEnd);

      let hit = false;
      if (startBig < endBig) {
        hit = targetBig > startBig && targetBig <= endBig;
      } else {
        // 跨越 0 点
        hit = targetBig > startBig || targetBig <= endBig;
      }

      if (hit) {
        return entry.resultNode;
      }
    }
  }

  return null;
}

// store(): LRU 驱逐 + 区间索引维护
store(
  targetId:      string,
  resultNode:    NodeInfo,
  intervalStart?: string,
  intervalEnd?:   string
): void {
  // LRU 驱逐
  if (this.map.size >= this.capacity && !this.map.has(targetId)) {
    const oldestKey = this.map.keys().next().value;
    if (oldestKey) this.evict(oldestKey);
  }

  const entry: CacheEntry = {
    targetId, resultNode,
    cachedAt: Date.now(),
    intervalStart, intervalEnd,
  };

  this.map.delete(targetId); // 先删，确保重新插到末尾
  this.map.set(targetId, entry);

  // 维护区间索引
  if (intervalEnd) {
    if (!this.intervalIndex.has(intervalEnd)) {
      this.intervalIndex.set(intervalEnd, []);
    }
    this.intervalIndex.get(intervalEnd)!.push(entry);
  }
}

private touch(key: string, entry: CacheEntry): void {
  this.map.delete(key);
```

```
    this.map.set(key, entry);
  }

  private evict(key: string): void {
    const entry = this.map.get(key);
    if (entry?.intervalEnd) {
      const list = this.intervalIndex.get(entry.intervalEnd);
      if (list) {
        const idx = list.indexOf(entry);
        if (idx !== -1) list.splice(idx, 1);
        if (list.length === 0) this.intervalIndex.delete(entry.intervalEnd);
      }
    }
    this.map.delete(key);
  }
}
```

第 11 章 Tracker Client 实现

> ***实现位置：*** *`src/tracker/tracker-client.ts`*
>
> ***功能职责：*** *TrackerClient 负责向 Tracker 注册节点、获取种子节点、上报心跳、拉取 CRL。具体职责包括：*
>
> - *注册（`register`）：节点 Join 时向 Tracker 上报自身 NodeInfo 与证书*
> - *获取种子（`getSeeds`）：从 Tracker 拉取已知活跃节点列表，用于 Join 引导*
> - *心跳（`heartbeat`）：周期性上报存活状态*
> - *注销（`deregister`）：Graceful Leave 时从 Tracker 移除自身*
> - *拉取 CRL（`fetchCRL`）：获取最新证书吊销列表*
>
> *（详细实现见配套实现文档）*

第 12 章 DO Storage 持久化策略

> ***持久化策略概述：***
>
> - ***写入时机：*** *状态变更后立即持久化（`persistState()`），保证 DO 驱逐后可完整恢复*
> - ***批量写入优化：*** *单次 `storage.put()` 写入整个 `PersistedState` JSON，避免多次 I/O*
> - ***冷启动恢复流程：*** *`blockConcurrencyWhile(initialize)` 确保首个请求到达前状态已恢复完毕*
> - ***内存缓存重建：*** *nonceCache、certCache、routingCache、rttCache 均为内存态，DO 驱逐后按需重建，协议自身容错*
>
> *（详细实现见配套实现文档）*

第 13 章 Graceful Leave：适配 API 触发

> ***触发方式：*** *通过 `POST /admin/leave` 接口触发*
>
> ***执行流程：***
>
> 1. *状态转移：`ACTIVE → LEAVING`*
> 2. *停止接受新 Chord 协议请求（返回 503）*
> 3. *向前驱通知后继变更*
> 4. *向后继传递前驱指针*
> 5. *从 Tracker 注销*
> 6. *状态转移：`LEAVING → INITIALIZING`，清空持久化状态*
>
> *（详细实现见配套实现文档）*

第 14 章 配置管理：Secrets 与环境变量

14.1 配置分类

配置项	存储方式	说明
`NODE_PRIVATE_KEY_BASE64`	`wrangler secret put`	Ed25519 私钥，32 字节，base64url 编码。不可写入 wrangler.toml
`NODE_CERTIFICATE_JSON`	`wrangler secret put`	完整 JSON 证书字符串，由 CA 签发
`CA_PUBLIC_KEY_BASE64`	`wrangler secret put`	CA 公钥，32 字节，base64url 编码，用于验证其他节点证书
`MANUAL_SEEDS`	`wrangler secret put`	JSON 数组字符串，手动指定种子节点，如 `'["https://node-b..."]'`
`NODE_URI`	`wrangler.toml [vars]`	本节点的 HTTPS URI，用于计算 node_id
`NODE_REGION`	`wrangler.toml [vars]`	地域标识，用于 v3.0 延迟感知路由评分（Region 亲和权重 0.1）
`TRACKER_URL`	`wrangler.toml [vars]`	Tracker 服务地址

配置项	存储方式	说明
AUTH_ENABLED	wrangler.toml [vars]	"true" 启用 v2.0 身份验证， "false" 跳过（测试用）

> ⚠️ **安全提示**
> 所有敏感配置（私钥、证书、CA 公钥、手动种子）必须通过 `wrangler secret put` 注入，**绝对不能**写入 `wrangler.toml` 或提交至版本控制系统。Secrets 在 Cloudflare 后端加密存储，仅在运行时通过 `env` 对象注入。

> *（详细配置校验与运行时读取逻辑见配套实现文档）*

第 15 章 Go → TypeScript 逐项映射对照表

Go 实现	TypeScript / CF Workers 对应	说明
sync.RWMutex	无需——DO 单线程串行执行	DO 天然消除所有并发锁
time.Ticker + goroutine	DO Alarm （ctx.storage.setAlarm()）	Alarm 自我续命形成永久循环
goroutine （并发任务）	Promise.allSettled(tasks)	fixFingersBatch 等并行操作
context.Context + 超时	AbortController + setTimeout	HTTP 请求超时控制
crypto/ed25519	crypto.subtle （Ed25519）	Web Crypto API，原生支持
crypto/sha1	crypto.subtle.digest("SHA-1")	计算节点 ID
math/big.Int （160-bit 环运算）	BigInt （原生）	JavaScript 原生大整数，无需依赖库
net/http 服务器	DO fetch() handler	请求在 DO 内串行处理
net/http 客户端	全局 fetch()（带签名）	通过 rpcFetch 封装自动签名
os.Getenv()	env.VAR_NAME	通过 wrangler.toml [vars] 或 Secrets 注入
内存状态（struct 字段）	DO 实例属性（private）	热路径，DO 存活期间常驻内存
文件持久化 / 数据库	DO Storage（SQLite）	ctx.storage.get/put，冷启动恢复
log.Printf	console.log	输出到 Cloudflare Workers 日志流

Go 实现	TypeScript / CF Workers 对应	说明
二进制部署	`wrangler deploy`	每次 deploy 对应一个节点版本

> *（完整逐函数映射对照表见配套实现文档）*

第 16 章 已知约束与边缘情况

16.1 已知约束汇总

约束项	影响	缓解措施
免费计划 DO CPU 30 ms 限制	Ed25519 验签 + 并发 fix_fingers 超限	升级 Workers Paid（$5/月），CPU 上限提升至 30 s
免费计划 subrequest 50 次 / 调用	fixFingersBatch(k=8) 并发受限	付费计划 1 000 次；或减小批量大小
DO 驱逐后内存缓存丢失	nonceCache、certCache、routingCache、rttCache 重置	协议本身容错；冷启动后缓存自动重建
DO 驱逐 nonce 缓存丢失	重放窗口期内理论上可重放	时间窗口（±5 min）限制重放范围；可接受的学习用途风险
单 Worker deployment = 单节点	多节点需多次 `wrangler deploy`	设计约束，符合 Chord 去中心化部署模型
同一 workers.dev 子域名冲突	测试环境两个节点共用同一 worker 名时 node_id 冲突	每个节点使用唯一 `name`（wrangler.toml）或自定义域名
DO Alarm 精度	Alarm 实际触发时间可能比预期晚数秒	维护周期对秒级抖动不敏感，可接受
BigInt 序列化	`JSON.stringify` 不支持 BigInt	所有 ID 均以 40 位 hex 字符串存储和传输，运算时临时转换
Ed25519 浏览器兼容性	Cloudflare Workers 运行时已全面支持	仅需注意本地测试环境（Node.js 18+ 支持）

> *（完整边缘情况分析与测试矩阵见配套实现文档）*
